

# Comparing Machine Learning Strategies for Quantum Noise Reduction

Amitav Krishna, Shanmukh Garimella

November 25, 2025

*This is currently a work in progress. The todos will be removed once the training has been run.*

## 1 Abstract

Running experiments on real quantum hardware remains extremely costly. Current cloud platforms charge between \$0.03 and \$0.05 per measurement, and practical algorithms often require tens of thousands of shots. Even modest experiments can therefore accumulate thousands of dollars in runtime fees, which places a hard barrier on iterative design and experimentation. Noise mitigation offers a way to extract higher quality results from the same number of measurements, reducing the number of measurements needed, and. This work presents a systematic comparison of four density matrix denoising strategies that pair two model classes, a convolutional autoencoder and a transformer autoencoder, with two loss formulations, a fidelity oriented reconstruction loss and a physics informed structural loss. We will be comparing how closely each model can return a pre-noise density matrix given the corresponding noisy density matrix.

- 1.1 **TODO** Once results come in, add a sentence or two discussing the empirical gains

## **2 Introduction**

- 2.1 **TODO** Add context on the state of ML-based noise reduction
- 2.2 **TODO** Add context on the current status quo for quantum error correction

## **3 Related implementations**

- 3.1 **TODO** Summarize prior ML denoising architectures

## **4 Methods**

### **4.1 Convolutional Autoencoder**

The convolutional autoencoder follows the architecture introduced by (Karan Kendre, 2025) for density-matrix denoising. The model treats each  $32 \times 32$  complex-valued density matrix as a two-channel image (real and imaginary components), enabling the network to exploit the local spatial patterns created by different quantum noise channels. The encoder consists of a sequence of convolutional blocks with ReLU activations and  $2 \times 2$  max-pooling, progressively reducing spatial resolution while expanding the channel dimension ( $1 \rightarrow 32 \rightarrow 64$ ). The decoder mirrors the encoder through nearest-neighbor up-sampling and transposed convolutions, reconstructing the full resolution density matrix. A final sigmoid activation constrains the reconstructed values to a normalized range.

### **4.2 Transformer Autoencoder**

The Transformer autoencoder models each  $32 \times 32$  complex-valued density matrix as a sequence of token embeddings, where each token corresponds to a row or column vectorized from the matrix's real and imaginary parts. This architecture allows every element in the input to attend globally to every other element, preserving the non-local structure characteristic of entangled quantum states. The encoder consists of a stack of Transformer encoder layers with multi-head self-attention, enabling the model to capture global correlations across the entire state. The encoded sequence is compressed

through a symmetric bottleneck module that projects the embedding dimension down and back up, serving as a latent representation. The decoder comprises Transformer decoder layers that apply both self-attention within the output sequence and cross-attention to the encoder memory. A linear projection and sigmoid activation generate the reconstructed sequence, which is reshaped into a 2D matrix.

### 4.3 Uhlmann fidelity-oriented reconstruction loss

The reconstruction loss incorporates the Uhlmann fidelity, the standard measure of similarity between mixed quantum states. For each predicted–target pair of density matrices  $(\hat{\rho})$  and  $(\rho)$ , the fidelity is computed by diagonalizing  $(\hat{\rho})$ , forming its matrix square root, and evaluating the trace of the geometric mean  $(\sqrt{\hat{\rho}}, \rho, \sqrt{\hat{\rho}})$ . This procedure captures discrepancies across eigenvalues and coherences, making it substantially more informative than elementwise losses when evaluating density-matrix reconstructions.

The Uhlmann fidelity is defined as

$$F(\hat{\rho}, \rho) = \left( \text{Tr} \sqrt{\sqrt{\hat{\rho}} \rho \sqrt{\hat{\rho}}} \right)^2$$

#### 4.3.1 TODO Switch out Uhlmann fidelity with frobenius reconstruction loss

#### 4.3.2 TODO Citation needed on that frobeniuis reconstruction loss

### 4.4 Physics-informed loss

The physics term enforces the basic structural axioms of a valid density matrix during reconstruction. Given a predicted state  $(\hat{\rho})$ , the loss penalizes violations of Hermiticity, trace normalization, and positive semidefiniteness, which are the three core constraints defining legitimate quantum states. Hermiticity is enforced by measuring the deviation between  $(\hat{\rho})$  and its conjugate transpose, trace preservation is encouraged by penalizing deviations of  $(\text{Tr}(\hat{\rho}))$  from unity, and positivity is promoted by penalizing negative eigenvalues of  $(\hat{\rho})$ . Together, these terms guide the model toward producing physically consistent outputs, irrespective of the noise model or architecture.

Formally,

$$\mathcal{L}_{physics}(\hat{\rho}) = \lambda_{\text{herm}} \left\| \hat{\rho} - \hat{\rho}^\dagger \right\|_2^2 + \lambda_{\text{trace}} (\text{Tr}(\hat{\rho}) - 1)^2 + \lambda_{\text{psd}} \left| \min(0, \lambda_i(\hat{\rho})) \right|_2^2,$$

where  $(\lambda_i(\hat{\rho}))$  are the eigenvalues of  $(\hat{\rho})$ . The total training loss combines this term with the Uhlmann fidelity loss

$$\mathcal{L}_{total} = F(\hat{\rho}, \rho) + \mathcal{L}_{physics}(\hat{\rho}).$$

#### 4.4.1 TODO Citation needed on that loss

### 4.5 Dataset

Clean states are generated by sampling random pure states and evolving them under randomly drawn single and two qubit unitaries. Each state is then corrupted by a stochastic noise channel that includes a mixture of depolarizing, amplitude damping, and phase damping noise with randomly sampled strengths. This produces paired datasets of noisy inputs and clean targets for supervised training. The full dataset contains one million simulated examples drawn uniformly from twenty noise type and noise level combinations.

#### 4.6 TODO Add link to circuit generator

#### 4.7 TODO Add expansion / citations for types of noise.

## 5 Experimental Design

We evaluate the four denoising configurations under a controlled and reproducible simulation pipeline designed to isolate the effects of architecture and loss function. All models are trained to map noisy density matrices to their clean counterparts using identical optimization settings. Training uses the Adam optimizer with a learning rate of  $1e4$ , batch size 64, and a fixed budget of 50 epochs. Weight initialization follows PyTorch defaults, and a validation split of ten percent of the training set is used for early stopping based on validation fidelity. Each model is trained from five different random seeds to account for stochastic variation, and reported results are averaged across these runs.

Evaluation is performed on the held out test set using multiple metrics. These include Uhlmann fidelity between the reconstructed and target states, Frobenius reconstruction error, trace deviation, Hermiticity deviation, and the magnitude of any negative eigenvalues. This combination of metrics allows us to assess both denoising performance and physical validity. Model capacities are kept comparable across architectures by matching parameter counts within ten percent.

## 5.1 Hyperparameters

### 5.1.1 Shared Hyperparameters

| Component        | Value               |
|------------------|---------------------|
| Optimizer        | Adam                |
| Learning rate    | 1e-4                |
| Weight decay     | 0                   |
| Batch size       | 64                  |
| Epochs           | 50                  |
| Early stopping   | Validation fidelity |
| Dataset size     | 100,000 circuits    |
| Train/test split | 80/20               |

### 5.1.2 Architecture-Specific Hyperparameters

| Component            | Convolutional Autoencoder                         | Transformer Autoencoder                   |
|----------------------|---------------------------------------------------|-------------------------------------------|
| Input representation | $32 \times 32 \times 2$                           | 1024 tokens (real/imag)                   |
| Embedding / channels | $2 \rightarrow 48 \rightarrow 96 \rightarrow 192$ | Linear $2 \rightarrow 32$                 |
| Encoder depth        | 3 conv blocks                                     | 4 layers, 4 heads                         |
| Decoder depth        | 3 conv blocks                                     | 4 layers                                  |
| Kernel size          | $3 \times 3$                                      | —                                         |
| Pooling              | MaxPool $2 \times 2$                              | —                                         |
| Upsampling           | Nearest neighbor                                  | —                                         |
| MLP hidden size      | —                                                 | 64                                        |
| Dropout              | 0.1 — 0.2                                         | 0.1                                       |
| Bottleneck           | —                                                 | Linear $32 \rightarrow 16 \rightarrow 32$ |
| Output projection    | Conv2d( $32 \rightarrow 2$ )                      | Linear $32 \rightarrow 2$                 |
| Activation           | ReLU, Sigmoid                                     | GELU, Sigmoid                             |
| Parameter Budget     | ~500k                                             | ~500k                                     |

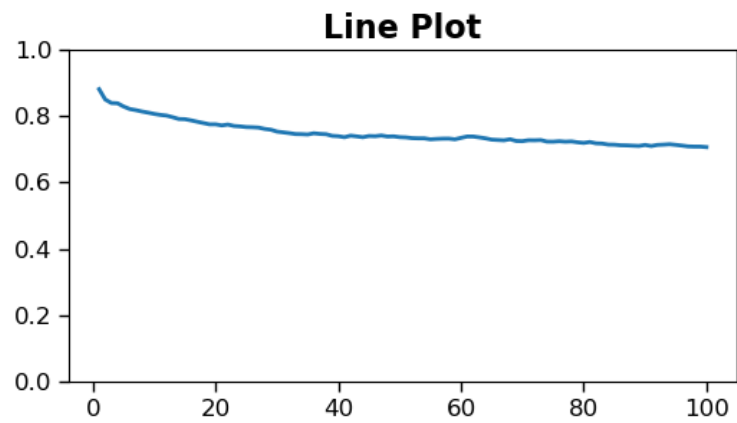
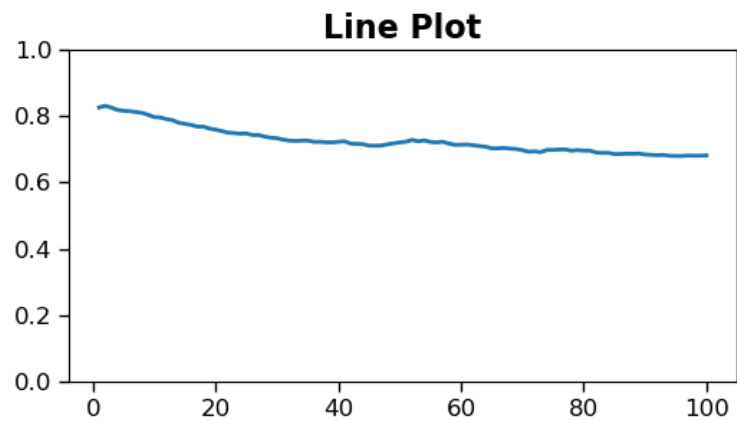
*We avoid patch embeddings in the Transformer to preserve element-level non-local structure; patching would impose a locality bias that conflicts with the architectural comparison.*

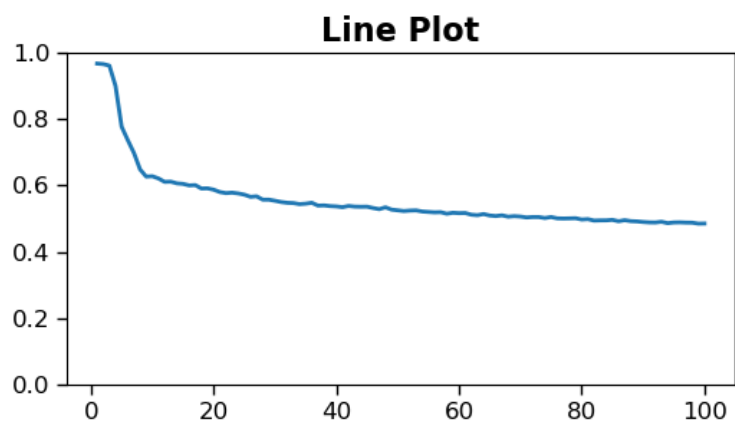
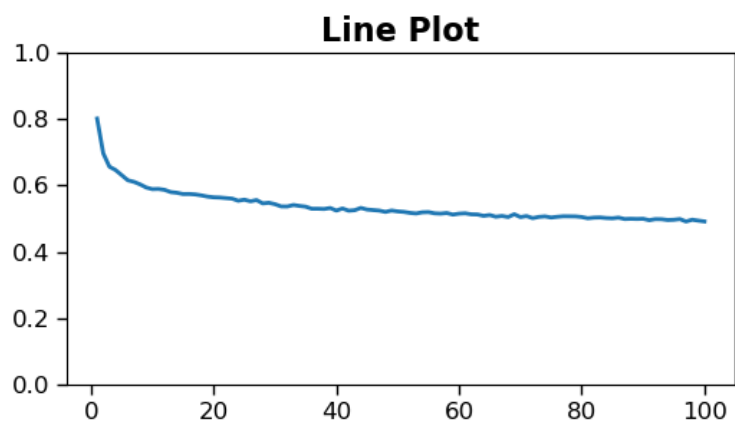
5.2 TODO Add details on compute used

5.3 TODO Add random-seed table or specify seed range

## 6 Results

6.1 Actual stuff (change this heading later)





#### 6.1.1 Quantitative Performance Comparison

| Model | Loss Type | Final Loss |
|-------|-----------|------------|
|-------|-----------|------------|

**6.1.2 Training Dynamics and Convergence Behavior**

**6.1.3 ARchitecture and Loss Function Comparison**

**6.1.4 Failure Modes and Qualitative Analysis**

**6.1.5 Variability Across Random Seeds**

## **7 Limitations**

## **8 Conclusion**

**8.1 TODO Add conclusion**

**8.2 TODO Talk about the scalabitlity of each approach**

## **9 References**

Karan Kendre (2025). *Machine Learning for Quantum Noise Reduction*.

**9.1 TODO Add more citations**